

Chapter 1

OOP in C# — Class Mechanics & Encapsulation

ENEL712 Embedded Systems Design — Lecture 15

Summary

Deeper dive into C# OOP: class mechanics (reference vs value), access modifiers, constructors and overloading, properties via get/set, and the Circle class evolution as a case study.

Topics Covered

- Introduction to OOP
- Encapsulation in detail
- Class mechanics in C#
- Constructors and overloading
- Properties: get and set
- Code example analysis (Circle class)

1.1 Introduction to OOP

- Programming paradigm using objects (classes) as modular building blocks.
- Each object has its own properties (data) and methods (functions).
- Designed to make large programs manageable; modules can be programmed/debugged/verified independently.
- Example: a bank account object — properties like balance, overdraft; methods like open, close, suspend.

1.2 Encapsulation in Detail

- Combine methods and properties into a single unit and control accessibility.

- Users of the class don't care how it works — they just want it to work (e.g., open a file).
- Internal data stays private — improves reliability and simplifies management.

1.3 Class Mechanics in C#

- Defined with the `class` keyword inside a namespace.
- Classes are reference types (cf. value types like `int`).
- Circle `c`; only creates an empty reference on the stack; memory is allocated on the heap with `new`.

1.3.1 Access modifiers

Modifier	Effect
private	Default; only the class itself can modify the data.
public	Accessible from outside the class.

1.4 Constructors and Overloading

- Constructor runs automatically on `new` — initialises properties.
- Same name as the class; no return type, not even `void`.
- Compiler generates a default constructor if you don't define one.
- Overloading: multiple methods/constructors with the same name but different parameter lists.

1.5 Properties: Get and Set

- Provide indirect, controlled access to private fields.
- `set` uses the implicit value parameter (e.g., `_radius = value;`).
- `get` returns the private field's value.
- Allow processing on read/write (e.g., unit conversion) and protect internal data.

1.6 Code Example: Circle Class Evolution

- Initial class: `public radius` — bad practice; lets external code change the field directly.
- Improved class: `private radius` enforces encapsulation.
- Final class: overloaded constructor (set radius on create) plus `get/set` properties — the recommended pattern.

1.7 Use Cases & Applications

- Sensor models: a `Sensor` class with a private calibration array exposes a clean `Read()` method while hiding the calibration math.

- Validation in setters: a `Temperature.Celsius` set { } block rejects values below absolute zero so bad data never reaches downstream code.
- Multiple constructors: a `Vector2D(double x, double y)` plus `Vector2D(double angle)` (with overloading) lets callers create the same object from different inputs.

1.8 Worked Example: Circle Class Evolution

1.8.1 Bad — public field

```
public class Circle
{
    public double radius;    // anyone can set this to -1
}
```

1.8.2 Better — private field

```
public class Circle
{
    private double radius;
    public Circle(double r) { radius = r; }
    public double Area() => Math.PI * radius * radius;
}
```

1.8.3 Best — encapsulated with property and validation

```
public class Circle
{
    private double radius;

    public Circle(double r) { Radius = r; }

    public double Radius
    {
        get => radius;
        set
        {
            if (value < 0) throw new ArgumentOutOfRangeException();
            radius = value;
        }
    }

    public double Area => Math.PI * radius * radius;
}
```

1.9 Stack vs Heap for a Class Instance

`Circle c = new Circle(2.5);` allocates as follows:

- Reference `c` lives on the stack (4–8 bytes).
- The `Circle` object (radius field, type info) lives on the managed heap.
- When `c` goes out of scope and no other reference exists, the GC can reclaim that heap memory.

1.10 Naming Conventions — Read at a Glance

Element	Convention	Example
Class / Type / Method	PascalCase	Circle, BankAccount, Open()
Public property	PascalCase	Radius, Balance
Local variable / parameter	camelCase	radius, accountId
Private field	<u>camelCase</u>	<u>radius</u> , <u>balance</u>
Constant	PascalCase or UPPER	Pi, MAX_SIZE
Namespace	PascalCase, dotted	MyCompany.Embedded.Drivers

- The leading underscore on a private field is a visual cue: 'this is internal, do not write to it from outside the class'.
- Using PascalCase for the public-facing Radius property and radius for its backing field lets callers use Radius while the class controls radius.
- Stick to one convention across the whole codebase — readers should be able to read intent from a name without checking the modifier.

1.11 Constructors — Default, Explicit, Overloaded

- If you do not write any constructor, the compiler generates a parameterless one that runs the default field initialisers (`int` $\rightarrow$ 0, `bool` $\rightarrow$ false, reference $\rightarrow$ null).
- Writing your own constructor replaces the implicit one — relying on default values is then your call to make explicit.
- Overloading lets you offer several construction shapes: parameterless, single-arg, copy constructor, etc.
- Once you write a constructor that takes parameters, the compiler-supplied no-arg constructor disappears unless you re-declare it.
- Constructor name = class name; no return type, not even void.

```
class Circle {
    private int _radius;

    // Explicit no-arg constructor -- initialises the field to a known value
    public Circle() { _radius = 0; }

    // Overload -- takes the radius
    public Circle(int r) { _radius = r; }

    public double Area() => Math.PI * _radius * _radius;
}

var a = new Circle(); // _radius = 0
var b = new Circle(4); // _radius = 4
```

1.12 The Circle Class — Three-Step Evolution

1.12.1 Step 1: public field — too open

```
class Circle {
    public int radius;                // any caller can mutate
    public double Area() => Math.PI * radius * radius;
}

var c = new Circle();
c.radius = -5;                       // nothing stops nonsense input
Console.WriteLine(c.Area());        // garbage out
```

External code can write any value, including invalid ones. There is no place to add validation or unit conversion later.

1.12.2 Step 2: private field + constructor — too closed

```
class Circle {
    private int _radius;
    public Circle(int r) { _radius = r; }
    public double Area() => Math.PI * _radius * _radius;
}

var c = new Circle(4);
// c._radius = 5; // compile error -- cannot access private field
// no way to change radius without a new Circle(...)
```

Encapsulated, but rigid — there is no controlled way to change the radius after construction.

1.12.3 Step 3: property with get/set — the gateway

```
class Circle {
    private int _radius;

    public Circle(int r) { _radius = r; }

    public int Radius {
        get { return _radius; }
        set {
            if (value < 0)
                throw new ArgumentOutOfRangeException(nameof(value));
            _radius = value;
        }
    }

    public double Area() => Math.PI * _radius * _radius;
}

var c = new Circle(4);
c.Radius = 7;           // goes through the set accessor -- validation runs
Console.WriteLine(c.Radius); // get accessor returns 7
```

- set runs whenever a caller writes to the property; the new value arrives in the implicit parameter named `value`.
- get runs whenever a caller reads the property.

- Either accessor can do work — validate input, lazy-load, raise an event, log, or simply return `_radius`.
- From the caller's perspective the property looks like a field, but the class keeps full control.

1.13 Properties as Gateways — Why Bother

For a single int the get/set wrapper looks like ceremony, but it pays off as the program grows.

- You can add validation in the setter without changing any caller.
- You can add unit conversion (set in metres, store in millimetres) transparently.
- You can refactor the backing storage (compute on the fly, lazy-load, cache) without breaking the public API.
- You can fire `INotifyPropertyChanged` or other events from the setter, enabling data binding.
- C# auto-properties (`public int Radius { get; set; }`) give you the gateway with zero ceremony — promote them to a full property only when behaviour is needed.

1.14 Method Overloading — Same Name, Different Signature

Overloading lets a single name describe a family of operations distinguished by their parameter list. The compiler picks the matching overload at the call site.

```
class Logger {
    public void Write(string message) { /* ... */ }
    public void Write(string message, int level) { /* ... */ }
    public void Write(Exception ex) { /* ... */ }
}

var log = new Logger();
log.Write("hello");                // Write(string)
log.Write("hello", 2);             // Write(string,int)
log.Write(new InvalidOperationException()); // Write(Exception)
```

- Overloads must differ in the number or type of parameters; differing only in return type is not enough.
- Constructors overload with the same rules.
- If two overloads are both applicable, the compiler picks the most specific match; ambiguities surface as compile errors.

Key Definitions

OOP

Paradigm structured around objects rather than functions and logic.

Class

Template/blueprint integrating methods and properties.

Object

Specific instance of a class created with `new`.

Encapsulation

Bundling methods and properties and restricting external access.

Property

Data field/member of an object.

Method

Function/action an object can perform.

Constructor

Automatically-invoked method that initialises a new object.

Overloading

Multiple methods/constructors with the same name but different parameter signatures.

Reference Type

Type whose variable holds a reference to data on the heap (e.g., class).

private

Access modifier limiting access to the defining class.

public

Access modifier allowing external access.

value (keyword)

Implicit parameter inside a property's set block.

Auto-Property

Shortcut C# syntax (public double Radius { get; set; }) where the compiler generates the backing field.

Expression-Bodied Member

Single-expression member written with $\Rightarrow$ (e.g., public double Area $\Rightarrow \pi * r * r$;

Defensive Setter

set block that validates the incoming value before storing it, preserving the object's invariants.

Auto-property

Shorthand syntax public T Name { get; set; } that lets the compiler synthesise the backing field.

Backing field

Private field that stores a property's value; conventionally named with a leading underscore (`_radius`).

value (keyword)

Implicit parameter inside a property setter; holds the value the caller is assigning.

Overloading

Defining multiple methods or constructors with the same name but different parameter lists.

Default constructor

Compiler-generated parameterless constructor that runs default field initialisers; suppressed once you declare any constructor of your own.

Sources

- Lecture 15.pdf